DOCUMENTAÇÃO DE ENGENHARIA DE REQUISITOS

Projeto: Assistente Inteligente RAG para Suporte MES

1. Introdução e Contextualização do Domínio

- **Descrição do Problema :** Dificuldade na rápida assimilação de termos técnicos, manuais operacionais extensos e regras de negócio complexas inerentes ao sistema MES (*Manufacturing Execution System*), o que gera dependência de suporte especializado de terceiro nível e impacta o tempo de resolução de ocorrências no chão de fábrica.
- **Abordagem Proposta:** Desenvolvimento de um assistente inteligente baseado em RAG (*Retrieval-Augmented Generation*), integrando busca vetorial com modelos de linguagem locais para centralizar, dinamizar e democratizar o acesso ao conhecimento técnico corporativo.
- **Fronteiras do Sistema:** O sistema abrange a indexação documental e a interface de consultas inteligentes em linguagem natural. Estão explicitamente fora do escopo a execução automática de comandos críticos de controle de máquinas e a tomada de decisão autônoma em cenários de emergência sem validação humana.

2. Elicitação e Análise de Requisitos

2.1. Requisitos Funcionais (RF)

- **RF01 - Consulta Textual em Linguagem Natural:** O sistema deve permitir que o usuário submeta perguntas em linguagem natural referentes a termos técnicos, falhas de processos e regras de negócio do MES.
- **RF02 - Recuperação de Contexto Vetorial:** O sistema deve consultar a base de dados vetorial (*PostgreSQL* com extensão *pgvector*) utilizando similaridade de cosseno para resgatar os trechos de manuais mais relevantes à dúvida apresentada.
- **RF03 - Geração de Respostas Aumentadas:** O sistema deve integrar os trechos recuperados a um modelo de linguagem local via *Ollama* para sintetizar uma resposta precisa e contextualizada, mitigando alucinações técnicas.
- **RF04 - Controle de Acesso e Segurança:** O sistema deve autenticar usuários e gerenciar permissões de acesso por meio de integração com *Keycloak*.
- **RF05 - Filtragem Contextual por Metadados:**O sistema deve permitir a aplicação de filtros prévios baseados em metadados (tais como código do equipamento, setor da fábrica e módulo do MES) antes da execução da busca

vetorial, garantindo que o contexto recuperado seja estritamente relevante ao escopo da consulta do usuário.

2.2. Requisitos Não Funcionais (RNF)

- **RNF01 - Segurança e Privacidade de Dados:** Todo o processamento e a inferência de IA devem ocorrer em ambiente local (*on-premise* via `Docker`), garantindo que dados industriais sensíveis não sejam expostos a APIs externas de terceiros.
- **RNF02 - Desempenho e Latência:** O tempo total de resposta da consulta não deve ultrapassar 5 segundos em condições normais de operação na rede interna.
- **RNF03 - Arquitetura e Portabilidade:** O backend da aplicação deve ser construído utilizando **Java (Quarkus)** com arquitetura modular, empacotado em contêineres Docker para facilitar o *deploy* e escalabilidade.
- **RNF04 - Manutenibilidade da Base Documental:** O pipeline de ingestão deve permitir atualizações incrementais e re-indexação de documentos técnicos (IOPs e manuais) sem interrupção dos serviços.

3. Visão Geral da Arquitetura Técnica

A arquitetura do projeto conecta componentes modernos de desenvolvimento *backend* em Java com inteligência artificial local:

- **Camada de Apresentação / API:** Microsserviços desenvolvidos em **Java (Quarkus)** expostos via REST.
- **Camada de Orquestração de IA: LangChain4j** gerenciando o fluxo de RAG e engenharia de *prompts*.
- **Camada de Persistência Vetorial: PostgreSQL com extensão pgvector** armazenando os *chunks* e *embeddings*.
- **Motor de Inferência Local: Ollama** executando modelos *open-source* de forma isolada e segura.

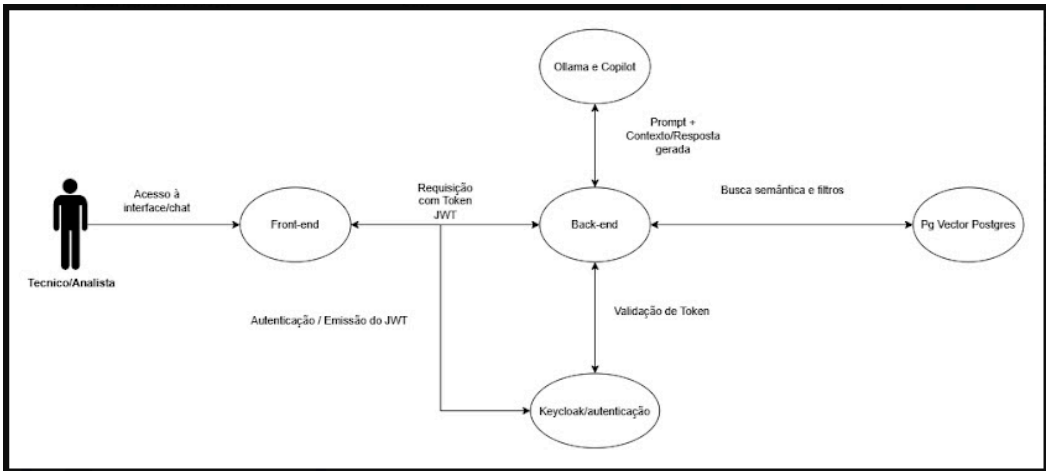


Figura 1: Arquitetura de componentes e fluxo de dados da solução RAG.

4. Estratégia de Dados e Modelagem (Data Strategy)

- **Fontes e Ingestão:** Manuais técnicos de equipamentos, Instruções de Operação de Processo (IOPs) e relatórios de turno extraídos do sistema MES.
- **Processamento e Vetorização:** Segmentação dos textos em *chunks* lógicos com sobreposição (*overlap*) e geração de representações semânticas através de um modelo de *embeddings* local.
- **Armazenamento e Metadados:** Persistência no PostgreSQL com extensão `pgvector`, associando atributos identificadores (como `equipment_code` e `factory_sector`) para permitir a filtragem contextual exigida nos requisitos funcionais.
-

5. Testes e Validação

- **Golden Dataset:** Definir a criação de um conjunto de perguntas reais feitas por técnicos do MES, mapeadas para as respostas perfeitas validadas por especialistas.
- **Métricas de Recuperação:** Avaliar se a busca do *pgvector* está realmente a encontrar a página certa do manual.
- **Métricas de Fidelidade:** Avaliar se o **Ollama** gerou a resposta baseada estritamente no manual, sem inventar passos de manutenção.

6. Implantação, Monitorização e Evolução (MLOps/LLMOps)

- **Infraestrutura (Deployment):** Especificar que será um ambiente *on-premise* isolado por questões de segurança, orquestrado via Docker e Docker Compose.
- **Monitorização (Monitoring):** Utilizar ferramentas (como o Quarkus Actuator) para controlar o consumo de recursos (CPU/GPU) do Ollama e registrar a latência das respostas (que devem demorar menos de 60 segundos).
- **Evolução (Feedback Loop):** Explicar como a base de dados será atualizada (ingestão incremental) sempre que um manual novo for lançado, e como os utilizadores poderão avaliar as respostas (polegar para cima/baixo) para melhoria contínua.

7. Matriz de Riscos e Mitigações

Risco Técnico	Descrição do Impacto	Estratégia de Mitigação na Arquitetura
Alucinação do Modelo de IA	O assistente inventar passos de manutenção que não existem nos manuais, colocando a operação em risco.	Uso estrito de RAG associado à **filtragem prévia por metadados**, limitando a busca ao equipamento ou setor correto. Adicionalmente, aplica-se ajuste de *temperature* (baixa) via LangChain4j e o modelo é instruído via *prompt* a responder "Não encontrei a informação" caso o contexto do *pgvector* seja insuficiente.
Vazamento de Dados Industriais	Exposição de regras de negócio, dados de produção ou segredos industriais em APIs públicas.	Arquitetura 100% *on-premise* isolada via contêineres Docker. Modelos executados localmente no Ollama, sem comunicação com a internet externa. Controle de acesso via Keycloak.
Respostas Desatualizadas	O técnico/analista receber uma instrução baseada num IOP (Instrução de Procedimento) antigo que já foi revogado.	Pipeline de ingestão incremental (descrito em MLOps) que atualiza o banco vetorial sempre que um novo manual for publicado, garantindo a rastreabilidade do documento mais recente.

Alta Latência (Lentidão)	O operador demorar muito tempo esperando a resposta da IA, atrasando a resolução do chamado na máquina.	Utilização de modelos *open-source* otimizados (como o Llama 3 8B) dimensionados para o hardware local, além da filtragem de metadados antes da busca vetorial para acelerar o processo.